

DDL Commands — CREATE, ALTER, DROP, TRUNCATE

Chapter 10 | *PostgreSQL Complete Guide* Module: 02_SQL_Basics | Difficulty: Beginner | Estimated Reading Time: 35 minutes

Table of Contents

- [Learning Objectives](#)
 - [Theory: What is DDL?](#)
 - [CREATE — Creating Database Objects](#)
 - [ALTER — Modifying Existing Objects](#)
 - [DROP — Removing Objects](#)
 - [TRUNCATE — Removing All Rows](#)
 - [ASCII Visual Diagrams](#)
 - [Practical Examples](#)
 - [PostgreSQL Commands](#)
 - [SQL Examples](#)
 - [Common Mistakes](#)
 - [Best Practices](#)
 - [Performance Considerations](#)
 - [Interview Questions](#)
 - [Interview Answers](#)
 - [Hands-on Exercises](#)
 - [Solutions](#)
 - [Advanced Notes](#)
 - [Cross-References](#)
-

Learning Objectives

By the end of this chapter, you will be able to:

- Distinguish DDL from DML, DCL, and TCL commands
 - Create tables, schemas, indexes, views, and sequences with proper options
 - Modify existing tables using ALTER TABLE (add/drop/rename columns, change types)
 - Safely drop database objects with proper checks
 - Understand the difference between DROP and TRUNCATE and when to use each
-

Theory: What is DDL?

DDL (Data Definition Language) is the subset of SQL used to **define, modify, and delete database structures** — schemas, tables, indexes, views, sequences, and other objects.

SQL Command Categories

Category	Full Name	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME, COMMENT	Define database structure

DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE, MERGE	Manipulate data
DCL	Data Control Language	GRANT, REVOKE	Manage permissions
TCL	Transaction Control Language	BEGIN, COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

DDL and Transactions in PostgreSQL

A critical PostgreSQL feature: **DDL statements are transactional.**

```
BEGIN;
CREATE TABLE test (id INT);
ALTER TABLE test ADD COLUMN name TEXT;
ROLLBACK; -- Table 'test' does not exist after this!
```

This is unique to PostgreSQL (and some other RDBMS). In MySQL and Oracle, DDL causes implicit commits and cannot be rolled back.

CREATE — Creating Database Objects

CREATE DATABASE

```
CREATE DATABASE myapp
ENCODING      = 'UTF8'
LC_COLLATE    = 'en_US.UTF-8'
LC_CTYPE      = 'en_US.UTF-8'
TEMPLATE      = template0
OWNER         = myapp_user;
```

CREATE SCHEMA

```
CREATE SCHEMA IF NOT EXISTS reporting;
CREATE SCHEMA catalog AUTHORIZATION app_user;
```

CREATE TABLE — Full Syntax

```
CREATE TABLE [IF NOT EXISTS] schema_name.table_name (
    column_name data_type [column_constraints],
    ...,
    [table_constraints]
);
```

Data Type Overview

Integer Types:	SMALLINT (2B), INTEGER/INT (4B), BIGINT (8B)
Auto-increment:	SMALLSERIAL, SERIAL, BIGSERIAL
Decimal:	NUMERIC(p,s), DECIMAL(p,s), REAL, DOUBLE PRECISION
Money:	MONEY (avoid; use NUMERIC instead)
Text:	CHAR(n), VARCHAR(n), TEXT
Boolean:	BOOLEAN
Date/Time:	DATE, TIME, TIMETZ, TIMESTAMP, TIMESTAMPTZ, INTERVAL
UUID:	UUID
JSON:	JSON, JSONB
Binary:	BYTEA
Network:	INET, CIDR, MACADDR
Array:	data_type[] (e.g., INTEGER[], TEXT[])
Range:	INT4RANGE, DATERANGE, TSRANGE, NUMRANGE
Geometric:	POINT, LINE, CIRCLE, POLYGON
Custom:	CREATE TYPE, CREATE DOMAIN, CREATE ENUM

Column Constraints

```
CREATE TABLE employees (
    emp_id      SERIAL          PRIMARY KEY,          -- PK: unique + not null
    email       VARCHAR(150)    UNIQUE NOT NULL,      -- unique, not null
    dept_id     INT             REFERENCES departments(id) ON DELETE SET NULL,
    salary      NUMERIC(10, 2)  CHECK (salary > 0),   -- check constraint
    status      VARCHAR(20)     DEFAULT 'active',     -- default value
    hire_date   DATE            NOT NULL DEFAULT CURRENT_DATE,
    manager_id  INT             REFERENCES employees(emp_id) -- self-ref
);
```

Table-Level Constraints

```
CREATE TABLE order_items (
    order_id    INT NOT NULL,
    product_id  INT NOT NULL,
    qty         INT NOT NULL CHECK (qty > 0),
    unit_price  NUMERIC(10, 2) NOT NULL,

    -- Table-level constraints (can reference multiple columns)
    CONSTRAINT pk_order_items PRIMARY KEY (order_id, product_id),
    CONSTRAINT fk_order       FOREIGN KEY (order_id) REFERENCES orders(id) ON
DELETE CASCADE,
    CONSTRAINT fk_product     FOREIGN KEY (product_id) REFERENCES products(id),
    CONSTRAINT chk_positive_price CHECK (unit_price >= 0)
);
```

CREATE INDEX

```
-- Basic B-tree index
CREATE INDEX idx_customers_email ON customers(email);
```

```

-- Unique index
CREATE UNIQUE INDEX idx_products_sku ON products(sku);

-- Multi-column composite index
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date DESC);

-- Partial index (only indexes rows matching a condition)
CREATE INDEX idx_orders_pending ON orders(created_at)
WHERE status = 'pending';

-- Index on expression
CREATE INDEX idx_customers_lower_email ON customers(LOWER(email));

-- GIN index for JSONB
CREATE INDEX idx_products_attrs ON products USING GIN(attrs);

-- GIN index for full-text search
CREATE INDEX idx_articles_tsv ON articles USING GIN(to_tsvector('english', body));

-- BRIN index for large tables with naturally ordered data
CREATE INDEX idx_events_time ON events USING BRIN(occurred_at);

-- Create index concurrently (doesn't block reads/writes)
CREATE INDEX CONCURRENTLY idx_users_created ON users(created_at);

```

CREATE VIEW

```

-- Simple view
CREATE VIEW active_customers AS
SELECT customer_id, first_name, last_name, email
FROM customers
WHERE status = 'active';

-- View with computed columns
CREATE VIEW order_summary AS
SELECT
    o.order_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    o.order_date,
    o.total,
    COUNT(oi.product_id) AS item_count
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, c.first_name, c.last_name, o.order_date, o.total;

-- Materialized view (stores results physically)
CREATE MATERIALIZED VIEW mv_daily_revenue AS
SELECT
    DATE_TRUNC('day', order_date) AS day,

```

```

        SUM(total) AS revenue
FROM orders
GROUP BY DATE_TRUNC('day', order_date)
WITH DATA;

-- Refresh materialized view
REFRESH MATERIALIZED VIEW mv_daily_revenue;
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_revenue; -- Non-blocking

```

CREATE SEQUENCE

```

CREATE SEQUENCE order_seq
  START 1000
  INCREMENT 1
  MINVALUE 1000
  MAXVALUE 9999999
  NO CYCLE;

-- Use in table
CREATE TABLE orders (
  order_id INT DEFAULT NEXTVAL('order_seq') PRIMARY KEY,
  ...
);

-- Next value
SELECT NEXTVAL('order_seq');

-- Current value (within current session)
SELECT CURRVAL('order_seq');

-- Reset sequence
ALTER SEQUENCE order_seq RESTART WITH 1000;

```

CREATE TYPE

```

-- Enum type
CREATE TYPE order_status AS ENUM ('pending', 'processing', 'shipped', 'delivered',
'cancelled');

-- Composite type
CREATE TYPE address_type AS (
  street TEXT,
  city TEXT,
  state CHAR(2),
  zip VARCHAR(10),
  country VARCHAR(50)
);

-- Domain (constrained type)

```

```
CREATE DOMAIN positive_int AS INTEGER CHECK (VALUE > 0);
CREATE DOMAIN email_address AS VARCHAR(200)
    CHECK (VALUE ~ '^[^\s]+@[^\s]+\.[^\s]+$');

-- Use types in tables
CREATE TABLE shipments (
    id          SERIAL PRIMARY KEY,
    status      order_status DEFAULT 'pending',
    ship_to     address_type,
    weight_kg   positive_int
);
```

ALTER — Modifying Existing Objects

ALTER TABLE — Add Column

```
-- Add a single column
ALTER TABLE customers ADD COLUMN phone VARCHAR(20);

-- Add with constraint and default
ALTER TABLE customers
    ADD COLUMN last_login TIMESTAMPTZ DEFAULT NOW();

-- Add NOT NULL column (must provide DEFAULT for existing rows)
ALTER TABLE products
    ADD COLUMN is_active BOOLEAN NOT NULL DEFAULT TRUE;
```

ALTER TABLE — Drop Column

```
-- Drop a column
ALTER TABLE customers DROP COLUMN phone;

-- Drop with cascade (also drops dependent views, indexes, etc.)
ALTER TABLE customers DROP COLUMN phone CASCADE;

-- Safe drop (won't error if column doesn't exist)
ALTER TABLE customers DROP COLUMN IF EXISTS phone;
```

ALTER TABLE — Rename

```
-- Rename a column
ALTER TABLE customers RENAME COLUMN phone TO phone_number;

-- Rename a table
ALTER TABLE customers RENAME TO clients;
```

```
-- Rename a constraint
ALTER TABLE orders RENAME CONSTRAINT orders_pkey TO pk_orders;
```

ALTER TABLE — Change Data Type

```
-- Change data type (may require USING clause for type conversion)
ALTER TABLE products
  ALTER COLUMN price TYPE NUMERIC(12, 2);

-- Change type with explicit conversion
ALTER TABLE orders
  ALTER COLUMN legacy_id TYPE BIGINT USING legacy_id::BIGINT;

-- Set/drop NOT NULL
ALTER TABLE customers ALTER COLUMN email SET NOT NULL;
ALTER TABLE customers ALTER COLUMN phone DROP NOT NULL;

-- Set/drop DEFAULT
ALTER TABLE orders ALTER COLUMN status SET DEFAULT 'pending';
ALTER TABLE orders ALTER COLUMN status DROP DEFAULT;
```

ALTER TABLE — Constraints

```
-- Add constraint
ALTER TABLE orders
  ADD CONSTRAINT chk_positive_total CHECK (total > 0);

ALTER TABLE employees
  ADD CONSTRAINT fk_dept
  FOREIGN KEY (dept_id) REFERENCES departments(id)
  ON DELETE SET NULL;

-- Drop constraint
ALTER TABLE orders DROP CONSTRAINT chk_positive_total;

-- Disable/enable trigger
ALTER TABLE orders DISABLE TRIGGER ALL;
ALTER TABLE orders ENABLE TRIGGER ALL;
```

ALTER INDEX

```
-- Rename index
ALTER INDEX idx_customers_email RENAME TO idx_cust_email;

-- Set tablespace
ALTER INDEX idx_customers_email SET TABLESPACE fast_ssd;
```

ALTER VIEW

```
-- Replace view definition (column names must match)
CREATE OR REPLACE VIEW active_customers AS
SELECT customer_id, first_name, last_name, email, phone
FROM customers
WHERE status = 'active' AND created_at > '2020-01-01';
```

DROP — Removing Objects

DROP TABLE

```
-- Drop a table
DROP TABLE customers;

-- Safe drop (no error if not exists)
DROP TABLE IF EXISTS customers;

-- Drop multiple tables
DROP TABLE orders, order_items;

-- Drop with CASCADE (also drops dependent views, FKs, etc.)
DROP TABLE customers CASCADE;

-- Drop without CASCADE (fails if dependencies exist)
DROP TABLE customers RESTRICT; -- RESTRICT is the default
```

DROP Other Objects

```
-- Drop database
DROP DATABASE IF EXISTS old_db;

-- Drop schema and all its objects
DROP SCHEMA reporting CASCADE;

-- Drop index
DROP INDEX IF EXISTS idx_customers_email;
DROP INDEX CONCURRENTLY idx_customers_email; -- Non-blocking

-- Drop view
DROP VIEW IF EXISTS active_customers CASCADE;

-- Drop materialized view
DROP MATERIALIZED VIEW IF EXISTS mv_daily_revenue;

-- Drop sequence
DROP SEQUENCE IF EXISTS order_seq;
```



```
-- Drop type
DROP TYPE IF EXISTS order_status CASCADE;

-- Drop domain
DROP DOMAIN IF EXISTS email_address;
```

TRUNCATE — Removing All Rows

```
-- Remove all rows (much faster than DELETE FROM table)
TRUNCATE TABLE orders;

-- Truncate multiple tables
TRUNCATE TABLE orders, order_items;

-- Truncate with RESTART IDENTITY (resets sequences)
TRUNCATE TABLE orders RESTART IDENTITY;

-- Truncate with CASCADE (also truncates referencing tables)
TRUNCATE TABLE customers CASCADE;

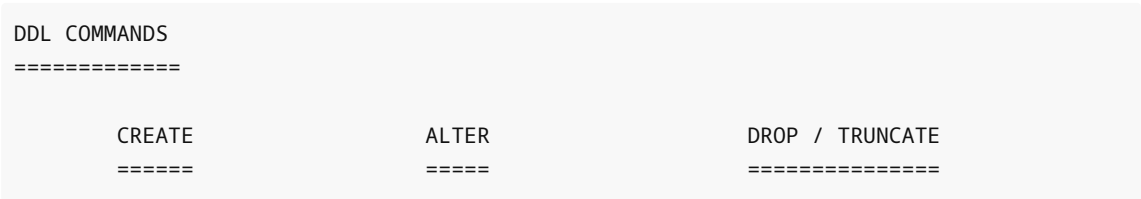
-- Truncate and reset sequences
TRUNCATE TABLE orders RESTART IDENTITY CASCADE;
```

TRUNCATE vs. DELETE

Feature	TRUNCATE	DELETE
Speed	Very fast (DDL)	Slow for large tables (DML)
WHERE clause	No	Yes
Triggers ON DELETE	No	Yes
Rollback	Yes (in PostgreSQL)	Yes
WAL generated	Minimal	Full row-level WAL
MVCC visibility	Not visible to concurrent transactions	Visible
Auto-increments	Can reset (RESTART IDENTITY)	Does not reset

ASCII Visual Diagrams

Diagram 1: DDL Command Hierarchy



CREATE DATABASE	ALTER TABLE	DROP TABLE [CASCADE]
CREATE SCHEMA	ALTER TABLE ... ADD COLUMN	DROP SCHEMA [CASCADE]
CREATE TABLE	ALTER TABLE ... DROP COLUMN	DROP INDEX [CONCURRENTLY]
CREATE INDEX	ALTER TABLE ... RENAME	DROP VIEW
CREATE VIEW	ALTER TABLE ... ALTER COLUMN	DROP SEQUENCE
CREATE MAT. VIEW	ALTER TABLE ... ADD CONSTRAINT	TRUNCATE TABLE
CREATE SEQUENCE	ALTER INDEX	TRUNCATE ... RESTART IDENTITY
CREATE TYPE	ALTER VIEW	
CREATE DOMAIN	ALTER SEQUENCE	
CREATE FUNCTION	ALTER TYPE	
CREATE TRIGGER	ALTER DATABASE	
CREATE EXTENSION	ALTER SCHEMA	

Diagram 2: Table Creation Flow

DESIGNING A TABLE =====

1. Identify entity --> customers, orders, products
|
2. Choose primary key --> SERIAL (auto-increment) or UUID
|
3. Define columns --> names, data types
|
4. Add NOT NULL constraints --> which columns must always have a value?
|
5. Add UNIQUE constraints --> email, SKU, username
|
6. Add CHECK constraints --> price > 0, age BETWEEN 0 AND 150
|
7. Add FOREIGN KEYS --> link to parent tables
|
8. Set DEFAULTs --> created_at = NOW(), status = 'active'
|
9. Add indexes --> on FKs, frequently queried columns
|
10. Add comments --> COMMENT ON TABLE/COLUMN

Diagram 3: ALTER TABLE Column Change Safety

SAFE ALTER OPERATIONS (No table rewrite):

- + ADD COLUMN with DEFAULT (PostgreSQL 11+)
- + ADD COLUMN nullable
- + DROP COLUMN (logical removal)
- + ADD/DROP CONSTRAINT
- + SET/DROP DEFAULT
- + RENAME COLUMN

UNSAFE ALTER OPERATIONS (Require full table rewrite):

- + ALTER COLUMN TYPE (may need to convert data)
- + ADD NOT NULL to existing column (without default)

```
+ ADD CHECK CONSTRAINT (must validate all rows)
```

PostgreSQL 11+ improvement:

```
ADD COLUMN ... NOT NULL DEFAULT value
```

```
--> No longer requires table rewrite if DEFAULT is immutable!
```

Practical Examples

Example 1: E-Commerce Schema Creation

```
-- Create the full e-commerce schema
CREATE SCHEMA ecommerce;

CREATE TABLE ecommerce.categories (
    category_id SERIAL PRIMARY KEY,
    name         VARCHAR(100) UNIQUE NOT NULL,
    parent_id    INT REFERENCES ecommerce.categories(category_id),
    description   TEXT
);

CREATE TABLE ecommerce.products (
    product_id  SERIAL PRIMARY KEY,
    sku         VARCHAR(50) UNIQUE NOT NULL,
    name        VARCHAR(200) NOT NULL,
    description  TEXT,
    price       NUMERIC(10, 2) NOT NULL CHECK (price >= 0),
    category_id INT REFERENCES ecommerce.categories(category_id),
    stock_qty   INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ DEFAULT NOW(),
    updated_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_products_category ON ecommerce.products(category_id);
CREATE INDEX idx_products_sku      ON ecommerce.products(sku);
CREATE INDEX idx_products_active   ON ecommerce.products(is_active) WHERE is_active
= TRUE;
```

PostgreSQL Commands

```
# List tables
\d
\d ecommerce.*

# Describe table structure
\d tablename
\d+ tablename      # with sizes and extra details
```

```
# List indexes
\di
\di+ tablename

# List sequences
\ds

# List views
\dv

# List types
\dT

# List schemas
\dn
```

SQL Examples

Example 1: CREATE TABLE with All Constraint Types

```
CREATE TABLE full_example (
  id          BIGSERIAL      PRIMARY KEY,
  uuid_col    UUID          DEFAULT gen_random_uuid() UNIQUE,
  name        VARCHAR(100)  NOT NULL,
  email       TEXT          UNIQUE NOT NULL,
  age         SMALLINT      CHECK (age BETWEEN 0 AND 150),
  salary      NUMERIC(12,2) CHECK (salary > 0),
  status      VARCHAR(20)   NOT NULL DEFAULT 'active',
  manager_id  BIGINT        REFERENCES full_example(id),
  metadata    JSONB,
  tags        TEXT[],
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  CONSTRAINT uq_name_email UNIQUE (name, email)
);
```

Example 2: CREATE TABLE AS (CTAS)

```
-- Create a table from a query result (structure + data)
CREATE TABLE high_value_customers AS
SELECT customer_id, first_name, last_name, total_spent
FROM customers
WHERE total_spent > 1000;

-- CTAS with no data (structure only)
CREATE TABLE customers_backup AS
SELECT * FROM customers WITH NO DATA;
```

Example 3: ALTER TABLE — Add Multiple Columns

```
ALTER TABLE products
  ADD COLUMN weight_kg NUMERIC(6, 3),
  ADD COLUMN dimensions JSONB,
  ADD COLUMN image_url TEXT;
```

Example 4: Change Column Type Safely

```
-- Convert VARCHAR to TEXT (widens, safe)
ALTER TABLE products ALTER COLUMN description TYPE TEXT;

-- Convert INT to BIGINT (widens, safe)
ALTER TABLE orders ALTER COLUMN order_id TYPE BIGINT;

-- Convert TEXT to INTEGER (requires USING)
ALTER TABLE legacy_data
  ALTER COLUMN amount TYPE NUMERIC(10,2)
  USING amount::NUMERIC(10,2);
```

Example 5: Add Constraint to Existing Table

```
-- Add NOT NULL (only works if no NULLs exist, or use SET DEFAULT first)
UPDATE products SET weight_kg = 0 WHERE weight_kg IS NULL;
ALTER TABLE products ALTER COLUMN weight_kg SET NOT NULL;
ALTER TABLE products ALTER COLUMN weight_kg SET DEFAULT 0;

-- Add CHECK constraint (validates all existing data)
ALTER TABLE products
  ADD CONSTRAINT chk_weight CHECK (weight_kg >= 0);

-- Add FOREIGN KEY
ALTER TABLE products
  ADD CONSTRAINT fk_category
  FOREIGN KEY (category_id) REFERENCES categories(id)
  ON DELETE SET NULL
  DEFERRABLE INITIALLY DEFERRED;
```

Example 6: Rename Operations

```
ALTER TABLE customers RENAME TO clients;
ALTER TABLE clients RENAME COLUMN phone TO phone_number;
ALTER INDEX idx_customers_email RENAME TO idx_clients_email;
ALTER SEQUENCE customers_customer_id_seq RENAME TO clients_id_seq;
```

Example 7: DROP with Safety Checks

```
-- Check before dropping
SELECT COUNT(*) FROM information_schema.tables
WHERE table_name = 'old_customers';

-- Safe drop
DROP TABLE IF EXISTS old_customers CASCADE;

-- Verify drop
SELECT table_name FROM information_schema.tables
WHERE table_name = 'old_customers'; -- Should return 0 rows
```

Example 8: TRUNCATE vs DELETE Comparison

```
-- Setup
CREATE TABLE test_data (id SERIAL, val TEXT);
INSERT INTO test_data (val) SELECT md5(gs::TEXT) FROM generate_series(1,100000) gs;

-- TRUNCATE: instant, resets SERIAL
TRUNCATE TABLE test_data RESTART IDENTITY;
SELECT COUNT(*) FROM test_data; -- 0

-- DELETE: slower, SERIAL not reset
INSERT INTO test_data (val) VALUES ('test');
DELETE FROM test_data;
INSERT INTO test_data (val) VALUES ('after delete');
SELECT id FROM test_data; -- Will NOT be 1 (ID continues from before delete)
```

Example 9: Temporal Table Pattern (Valid From/To)

```
CREATE TABLE product_prices (
    product_id INT NOT NULL REFERENCES products(id),
    price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    valid_from DATE NOT NULL DEFAULT CURRENT_DATE,
    valid_to DATE,
    CONSTRAINT no_overlap EXCLUDE USING GIST (
        product_id WITH =,
        DATERANGE(valid_from, valid_to, '[') WITH &&
    )
);
```

Example 10: CREATE OR REPLACE

```
-- Replace view without dropping (column list must be compatible)
CREATE OR REPLACE VIEW active_products AS
SELECT product_id, name, price, stock_qty, updated_at
FROM products
WHERE is_active = TRUE AND stock_qty > 0;
```

```
-- Create function (replace if exists)
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trg_products_updated
BEFORE UPDATE ON products
FOR EACH ROW
EXECUTE FUNCTION update_updated_at();
```

Example 11: COMMENT ON Objects

```
COMMENT ON TABLE products IS 'Master product catalog for the e-commerce platform';
COMMENT ON COLUMN products.sku IS 'Stock Keeping Unit - unique product identifier
for inventory';
COMMENT ON COLUMN products.price IS 'Selling price in USD; never store in MONEY
type';
COMMENT ON INDEX idx_products_category IS 'Supports category-based product browsing
queries';
```

Example 12: Check Object Dependencies Before Dropping

```
-- Find objects that depend on a table before dropping
SELECT
    dependent.relname AS dependent_object,
    dep.deptype       AS dependency_type
FROM pg_depend dep
JOIN pg_class dependent ON dep.objid = dependent.oid
JOIN pg_class source    ON dep.refobjid = source.oid
WHERE source.relname = 'customers'
    AND dep.deptype = 'n'; -- 'n' = normal dependency
```

Common Mistakes

Mistake 1: Dropping Tables Without IF EXISTS

```
-- BAD: Fails with error if table doesn't exist
DROP TABLE old_customers;

-- GOOD: Safe, idempotent
DROP TABLE IF EXISTS old_customers;
```

Mistake 2: ALTER COLUMN TYPE Without USING

```
-- BAD: May fail with "cannot cast type"
ALTER TABLE orders ALTER COLUMN amount TYPE BIGINT;

-- GOOD: Provide explicit cast
ALTER TABLE orders ALTER COLUMN amount TYPE BIGINT USING amount::BIGINT;
```

Mistake 3: TRUNCATE Without RESTART IDENTITY

After TRUNCATE, SERIAL sequences continue from where they left off unless you use RESTART IDENTITY. This surprises developers who expect IDs to reset.

Mistake 4: Not Using IF NOT EXISTS for CREATE

```
-- BAD: Error if table already exists
CREATE TABLE customers (...);

-- GOOD: Idempotent (useful in migration scripts)
CREATE TABLE IF NOT EXISTS customers (...);
```

Mistake 5: Forgetting to Index Foreign Keys

```
-- BAD: FK without index causes slow JOINS
ALTER TABLE orders ADD CONSTRAINT fk_customer
    FOREIGN KEY (customer_id) REFERENCES customers(id);

-- GOOD: Always add index after FK
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
```

Mistake 6: Using TRUNCATE ... CASCADE in Production

`TRUNCATE TABLE customers CASCADE` will silently delete all orders, order_items, reviews — anything with an FK to customers. Always double-check cascade targets.

Mistake 7: Creating Indexes on Low-Cardinality Columns

```
-- BAD: Boolean column has only 2 values; B-tree index rarely helps
CREATE INDEX idx_is_active ON products(is_active);

-- GOOD: Partial index (only indexes 'true' rows, far fewer)
CREATE INDEX idx_active_products ON products(product_id) WHERE is_active = TRUE;
```

Best Practices

1. Use `IF NOT EXISTS` and `IF EXISTS` in all CREATE/DROP statements to make scripts idempotent (safe to re-run).
2. Name all constraints — `CONSTRAINT pk_orders PRIMARY KEY` gives clear error messages.

3. **Always index foreign key columns** — PostgreSQL does not auto-index FKs.
4. **Use `CREATE INDEX CONCURRENTLY`** in production to avoid blocking reads and writes.
5. **Use schemas** to organize tables: `CREATE TABLE billing.invoices` instead of dumping everything in `public`.
6. **Use `TIMESTAMPTZ` not `TIMESTAMP`** for all datetime columns.
7. **Add `COMMENT ON TABLE` and `COLUMN`** to document non-obvious design decisions.
8. **Test ALTERs on a copy** before running on production, especially type changes on large tables.
9. **Use `TRUNCATE RESTART IDENTITY`** when you need to truly reset a table.
10. **Version-control all DDL** — use migration tools (Flyway, Liquibase, Alembic).

Performance Considerations

- **Large table column addition:** PostgreSQL 11+ can add `NOT NULL DEFAULT` columns without a table rewrite (instant for large tables)
- **Type changes** on large tables require a full table rewrite — plan for downtime or use triggers
- `CREATE INDEX CONCURRENTLY` takes longer but doesn't block queries
- `DROP INDEX CONCURRENTLY` drops the index without blocking reads
- `TRUNCATE` is faster than `DELETE FROM table` for removing all rows (no per-row WAL, no triggers)
- `CREATE TABLE AS SELECT` is efficient for creating summary tables

```
-- Check table size before and after DDL operations
SELECT pg_size_pretty(pg_total_relation_size('orders')) AS size;
```

Interview Questions

1. What does DDL stand for and what commands does it include?
2. What is the difference between `DROP TABLE` and `TRUNCATE TABLE` ?
3. Can DDL be rolled back in PostgreSQL?
4. What happens if you `DROP TABLE customers CASCADE` ?
5. How do you add a `NOT NULL` column to an existing large table efficiently?
6. What is the difference between `DELETE` and `TRUNCATE` for removing all rows?
7. How do you rename a column in PostgreSQL?
8. What is `CREATE INDEX CONCURRENTLY` and why would you use it?
9. How do you check what objects depend on a table before dropping it?
10. What is a materialized view and how does it differ from a regular view?

Interview Answers

Q3: DDL Rollback in PostgreSQL

Yes — PostgreSQL is unique in that DDL statements are fully transactional. `CREATE TABLE` , `ALTER TABLE` , `DROP TABLE` can all be wrapped in a `BEGIN/ROLLBACK` block. If any DDL fails in a multi-statement

transaction, the entire transaction rolls back. This is very useful for migration scripts. Note: MySQL and Oracle DDL causes implicit commits and cannot be rolled back.

Q5: Adding NOT NULL Column to Large Table

In PostgreSQL 11+, `ALTER TABLE t ADD COLUMN col TEXT NOT NULL DEFAULT 'value'` where the default is an immutable constant does NOT require a table rewrite — it's nearly instant. Before PostgreSQL 11, this required rewriting the whole table. For older versions or when the default is non-immutable: (1) add column as nullable, (2) backfill in batches, (3) add NOT NULL constraint.

Q10: Materialized View vs. Regular View

A **view** is a saved query — every time you select from a view, the underlying query executes. No data is stored. A **materialized view** stores the query results as actual data on disk. Reads are fast (no re-execution) but data can be stale until `REFRESH MATERIALIZED VIEW` is called. Materialized views can have indexes, making them excellent for pre-computing expensive aggregations.

Hands-on Exercises

Exercise 1: Full Schema Creation

Create a schema called `inventory` with tables: `warehouses`, `products`, `stock_levels`. Include proper PKs, FKs, constraints, and indexes.

Exercise 2: ALTER TABLE Practice

Start with a simple `users` table. Add 5 columns one by one with different constraints. Rename one column. Change one column's data type. Drop one column.

Exercise 3: View and Materialized View

Create a view that shows products with their category names. Create a materialized view that shows total stock per category. Refresh it.

Exercise 4: Safe Migration Script

Write a migration script that adds a `discount_pct` column to the `products` table, backfills existing rows with 0, then adds a NOT NULL constraint. Make the script idempotent.

Exercise 5: Cleanup Script

Write a script that drops all objects in the `inventory` schema (tables, indexes, sequences) safely, handling the case where they may not exist.

Solutions

Solution 1

```
CREATE SCHEMA IF NOT EXISTS inventory;

CREATE TABLE inventory.warehouses (
  warehouse_id SERIAL PRIMARY KEY,
  name          VARCHAR(100) UNIQUE NOT NULL,
```

```

        location      TEXT,
        capacity      INT CHECK (capacity > 0)
    );

CREATE TABLE inventory.products (
    product_id SERIAL PRIMARY KEY,
    sku         VARCHAR(50) UNIQUE NOT NULL,
    name        VARCHAR(200) NOT NULL,
    unit_cost   NUMERIC(10,2) CHECK (unit_cost >= 0)
);

CREATE TABLE inventory.stock_levels (
    warehouse_id INT REFERENCES inventory.warehouses(warehouse_id) ON DELETE
CASCADE,
    product_id   INT REFERENCES inventory.products(product_id)      ON DELETE
CASCADE,
    qty          INT NOT NULL DEFAULT 0 CHECK (qty >= 0),
    updated_at   TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (warehouse_id, product_id)
);

CREATE INDEX idx_stock_product  ON inventory.stock_levels(product_id);
CREATE INDEX idx_stock_warehouse ON inventory.stock_levels(warehouse_id);

```

Solution 4

```

-- Idempotent migration
DO $$
BEGIN
    -- Add column if it doesn't exist
    IF NOT EXISTS (
        SELECT 1 FROM information_schema.columns
        WHERE table_name = 'products' AND column_name = 'discount_pct'
    ) THEN
        ALTER TABLE products ADD COLUMN discount_pct NUMERIC(5,2);
    END IF;
END $$;

-- Backfill
UPDATE products SET discount_pct = 0 WHERE discount_pct IS NULL;

-- Add NOT NULL
ALTER TABLE products ALTER COLUMN discount_pct SET NOT NULL;
ALTER TABLE products ALTER COLUMN discount_pct SET DEFAULT 0;

-- Add check constraint
ALTER TABLE products
    DROP CONSTRAINT IF EXISTS chk_discount;
ALTER TABLE products
    ADD CONSTRAINT chk_discount CHECK (discount_pct BETWEEN 0 AND 100);

```

Advanced Notes

DDL Locking

DDL commands acquire `ACCESS EXCLUSIVE` locks on affected tables, blocking ALL other operations. In busy production databases:

- Use maintenance windows for heavy ALTER TABLE operations
- Use `CREATE INDEX CONCURRENTLY` and `DROP INDEX CONCURRENTLY`
- Use `pg_repack` extension for live table reorganization without long locks

Generated Columns

```
CREATE TABLE orders (  
    qty          INT NOT NULL,  
    unit_price NUMERIC(10,2) NOT NULL,  
    total        NUMERIC(12,2) GENERATED ALWAYS AS (qty * unit_price) STORED  
);
```

Partitioned Tables

```
CREATE TABLE events (  
    event_id    BIGSERIAL,  
    occurred    TIMESTAMPTZ NOT NULL,  
    event_type  TEXT  
) PARTITION BY RANGE (occurred);  
  
CREATE TABLE events_2024 PARTITION OF events  
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

Cross-References

- **Previous:** [01 Fundamentals/09 installation guide.md](#)
- **Next:** [02 dml commands.md](#)
- **Related:** [01 Fundamentals/03 relational databases.md](#) — Keys and constraints
- **See Also:** [04 Advanced SQL/01 window functions.md](#)
- **See Also:** [07 Performance/01 indexing.md](#) — Complete indexing guide
- **See Also:** [05 Schema Design/01 normalization.md](#)